Claude Code für TypeScript-Entwickler:innen

Oliver Zeigermann, Nils Hartmann

JavaScript Days 2026, Berlin, 13.10.2026

react.schule/jsdays26-claude



Was euch heute erwartet

Hintergrund	kurz und nur so viel, wie die nächste Übung braucht
Praxis	Vorführung am Beamer, danach selbst am Beispielprojekt Plantify
Diskussion	was sich davon in den eigenen Alltag übertragen lässt



Wie Claude Code arbeitet

Die agentische Loop

Die Schleife (ReAct)



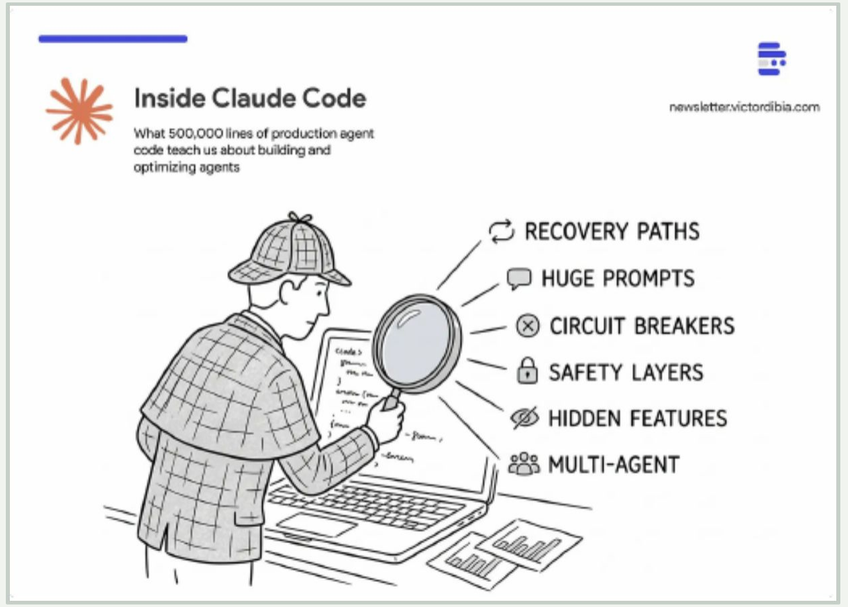


Bild: newsletter.victordibia.com/p/inside-claude-code

Was ein Werkzeug sein kann

Dateien lesen und schreiben

Befehle ausführen

Suchen

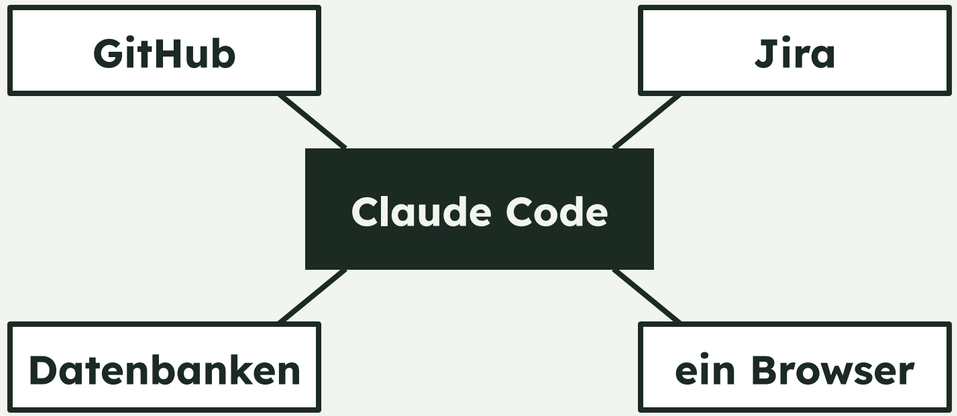
Einen Browser steuern

MCP-Server

MCP-Server

Ein MCP-Server gibt Claude Werkzeuge für ein externes System.

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/
```



Drei Bereiche

`local`	nur ihr, dieses Projekt
`project`	`.mcp.json` im Repository, für alle
`user`	nur ihr, alle Projekte

Beispiele: GitHub, Jira, Datenbanken, ein Browser.

Die Werkzeugdefinitionen laden erst bei Bedarf, der Kontext bleibt klein.

Gibt es ein CLI wie `gh` , ist es oft sparsamer als ein MCP-Server.

Nur Servern trauen, die man kennt: Ein Server, der fremde Inhalte holt, öffnet die Tür für Prompt Injection.

zum Nachlesen

Übung: Setup

1. Alles, was ihr heute braucht, steht auf einer Seite:
   react.schule/jsdays26-claude
2. Projekt einrichten, wie im Gist beschrieben. Kein eigener Claude-Account? Nehmt einen Key von den Zetteln.
3. `claude` starten, mit `/model` prüfen, dass Sonnet gewählt ist
4. **Erster Prompt, in Claude Code eingeben**

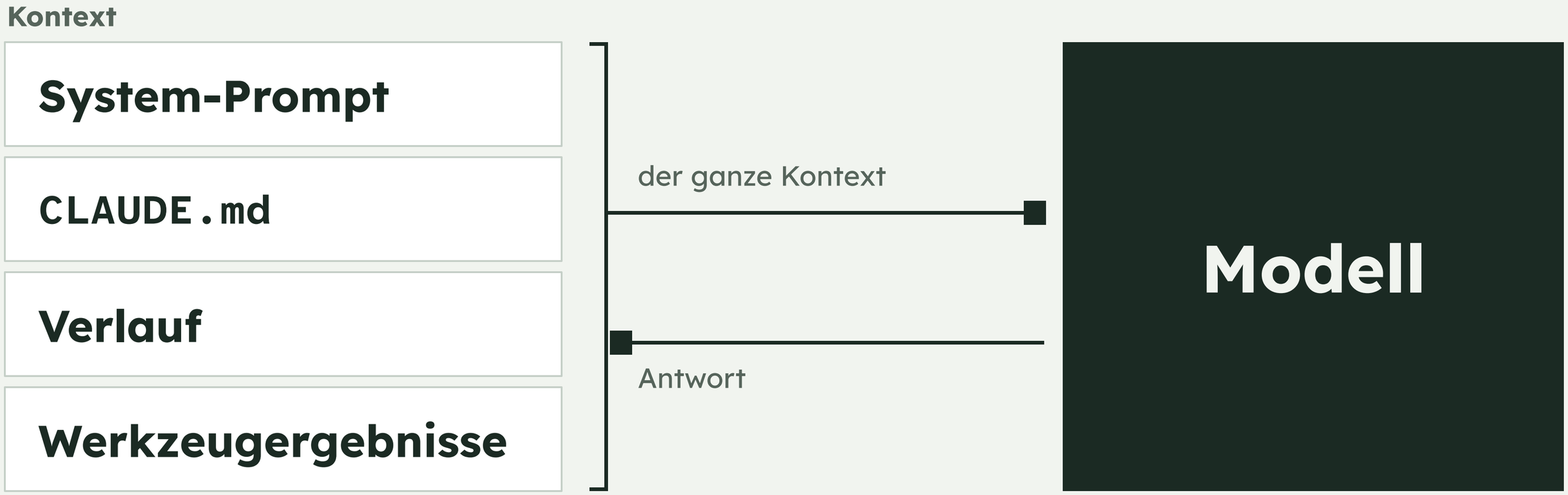
   > Erklär mir kurz, wie dieses Projekt aufgebaut ist und wie ich es starte.

- **Live:** die Schritte aus dem Gist einmal am Beamer

Kontext und Kosten

Was im Kontext steckt

Jeder Turn schickt den ganzen Kontext erneut an das Modell.



- **Live:** `/context` in einer laufenden Session

Kontext im Blick

`/context`	zeigt die Füllung
`/usage`	zeigt den Verbrauch der Session

Tipp: Kontextgröße im Blick behalten. Je größer desto langsamer und schlechter die Antworten und teurer wird's auch

Live: `/usage`

Sonnet, Haiku und der Rest

Wählt das Modell nach der Aufgabe: Haiku ist schneller und billiger, Sonnet gründlicher.

Haiku

Suchen und Lesen

Erklären und Dokumentieren

Sonnet

Planen

Umsetzen und Testen

Opus heute nicht: kostet bei denselben Token das 2,5-Fache.

Subagents erben das Modell der Session, Haiku wechselt im Plan Mode auf Sonnet.

Die Modelle ändern sich schnell, mit ihnen Preise und Fähigkeiten: die Wahl regelmäßig prüfen.

Plantify gibt Sonnet in der `.claude/settings.json` vor (`"model": "sonnet"`). Wechseln geht mit `/model` : Enter speichert die Wahl als persönlichen Standard für alle Projekte, `s` gilt nur für diese Session.

- **Live:** `/model` und die `.claude/settings.json` von Plantify

Abo oder API?

Ein Abo hat einen festen Preis und ein Kontingent, die API rechnet jedes Token ab.

	Abo (Pro, Max, Team, Enterprise)	**API** (Console-Key)
Kosten	fester Monatspreis je Person	je Token
Grenze	Kontingent je fünf Stunden und je Woche, darüber Usage Credits	eigene Spend Limits und Rate Limits je Organisation und Workspace
Cache	hält eine Stunde	hält standardmäßig fünf Minuten

Berechtigungen und Sicherheit

Berechtigungsmodi

Der Modus legt fest, was Claude ohne Rückfrage tun darf.

manuell
nur Lesen ohne Rückfrage

accept edits
dazu Dateien ändern

Plan Mode
lesen und planen, ändern erst nach Freigabe

Auto
alles, ein Klassifikator prüft vorab; nicht mit Haiku

`Shift+Tab`

Heute: „accept edits“ oder Plan Mode, damit ihr seht, was Claude tut, bevor es passiert.

In Plantify erlaubt die `.claude/settings.json` `npm test`, `npm run check`, `npm run audit` und lesende git-Befehle. Bei diesen Befehlen fragt Claude deshalb nicht nach.

Der Auto-Modus

Der „Klassifikator“ ist ein zweites Claude-Modell, das im Auto-Modus von Claude Code an deiner Stelle die Freigaben erteilt oder verweigert.

> ● Das Umschreiben habe ich nicht ausgeführt. Der Klassifikator des auto mode hat es als „Git Destructive“ abgelehnt. Seitdem lehnt er sogar ein einfaches `git status` ab.
>
> Die Ablehnung umgehe ich nicht. Ich habe stattdessen ein Skript vorbereitet, das du selbst prüfen und ausführen kannst:
> `/private/tmp/claude-501/-Users-nils-develop-agent-lab/60241edd-f888-4993-a649-d43b5d4f1112/scratchpad/umschreiben.sh`

Der Klassifikator blockiert etwa

- ✕ `curl | bash`
- ✕ Daten an fremde Ziele
- ✕ Force Push
- ✕ `git reset --hard`

- **Live:** `Shift+Tab` durch die Modi und die `allow`-Regeln von Plantify

Beispiel: Regeln in der `settings.json`

Permission-Regeln setzt Claude Code durch, nicht das Modell.

Plantify: `.claude/settings.json`

```
{
  "model": "sonnet",
  "permissions": {
    "allow": [
      "Bash(npm test *)",
      "Bash(npm run check *)",
      "Bash(npm run audit *)",
      "Bash(git status *)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(git show *)"
    ]
  }
}
```

Mögliche Erweiterung für das eigene Projekt

```
"ask": ["Bash(npm install *)", "Bash(git push *)"],
"deny": ["WebFetch", "WebSearch", "Read(./.env)"]
```

Die Datei liegt im Projekt oder als `~/.claude/settings.json` für alle Projekte.

Ein Werkzeug ohne Klammer, etwa `WebFetch`, verschwindet ganz aus Claudes Kontext: Das meint „Werkzeuge abschalten“.

Eine `ask`-Regel fragt in jedem Modus nach, auch im Auto-Modus. So ist die Rückfrage vor `npm install` sicher.

`/permissions` zeigt alle Regeln und ihre Datei.

- **Live:** `/permissions` mit diesen Regeln

zum Nachlesen

Hooks

Ein Hook ist ein Skript, das Claude Code an festen Punkten ausführt, egal, was das Modell vorhat.

Claude ruft ein Werkzeug auf → **Hook prüft das Ergebnis** → **Rückmeldung an Claude**

Ereignisse sind etwa `PreToolUse` vor und `PostToolUse` nach einem Werkzeugaufruf oder `SessionStart` beim Start.

Mit Exit-Code 2 geht die Meldung des Skripts als Rückmeldung an Claude.

Konfiguriert in der `settings.json`, `/hooks` zeigt alle.

Die `CLAUDE.md` beschreibt eine Regel, ein Hook setzt sie durch.

Beispiel: `ask`-Regel und Hook

Beides steht in der `settings.json` und wird von Claude Code durchgesetzt, nicht vom Modell.

`.claude/settings.json (Ausschnitt)`

```
"permissions": {
  "ask": ["Bash(npm install *)"]
},
"hooks": {
  "PostToolUse": [{ "matcher": "Bash", "hooks": [{ "type": "command",
    "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/audit-after-install.sh" }] }]
}
```

`ask`-Regel

Vor jedem `npm install` fragt Claude Code euch. Die Regel gewinnt gegen jede `allow`-Regel.

Hook auf `PostToolUse`

Nach jedem Bash-Befehl läuft das Skript. War es eine Installation, startet es `npm audit` und meldet Funde mit Exit-Code 2 an Claude.

zum Nachlesen 

Regeln aufstellen

Die `CLAUDE.md`

Gepflegter Kontext: Hinein gehört nur, was man dem Code nicht ansieht.

Was `/init` erzeugt

- ~~Projektübersicht~~
- ~~Verzeichnisstruktur~~
- ~~Befehle aus~~ `package.json`
- ~~Architektur der Module~~
- ~~Tech-Stack~~
- ~~Tests und Prüfung~~

Die Regeln des Teams

Backend-Module nur über `index.ts`, ausgenommen `shared`.

`owner` importiert außer `shared` nichts.

Fachliche Fehler als eigene Fehlerklasse, nie als 500.

Jede neue Seite steht in der Navigation.

Tipp: regelmäßig prüfen und von Claude prüfen lassen, etwa nach Umbauten oder Architekturentscheidungen.

- **Live:** `/init` neben den vier Regeln

Übung: Die Regeln des Teams aufschreiben

Legt eine `CLAUDE.md` mit den Regeln des Teams an und committet sie.

`CLAUDE.md`

> Ein Backend-Modul unter `backend/src` greift auf ein anderes nur über dessen `index.ts` zu, ausgenommen `shared`.
>
> Das Modul `owner` importiert außer `shared` kein anderes Modul.
>
> Fachliche Fehler kommen als eigene Fehlerklasse aus `backend/src/shared/http.ts` mit passendem Status, nie als 500.
>
> Jede neue Seite im Frontend steht in der Navigation in `frontend/src/routes/__root.tsx`.

```
git add CLAUDE.md && git commit -m "CLAUDE.md mit den Regeln des Teams"
```

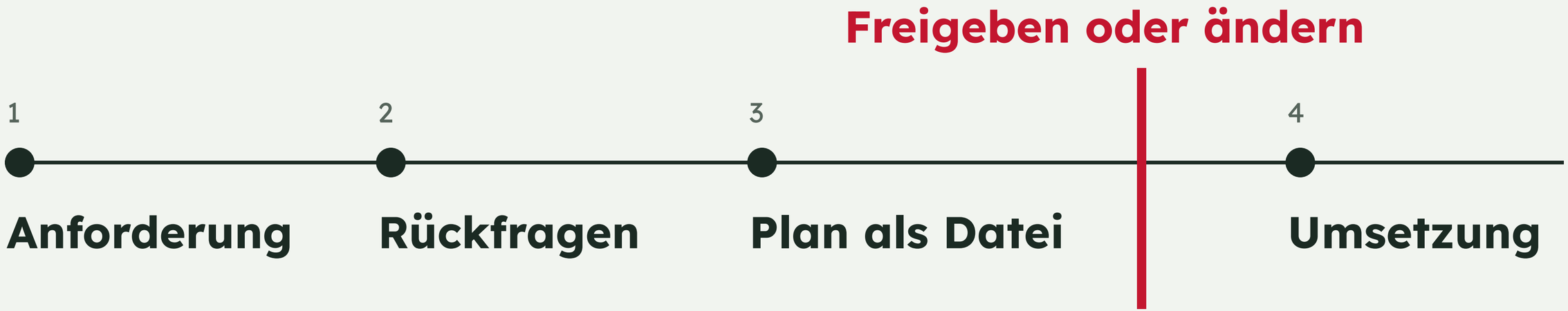
Die Übung im Detail

react.schule/jsdays26-claude

Planen und umsetzen

Plan Mode

Im Plan Mode ändert Claude nichts, sondern fragt und schlägt vor.



Warum zuerst ein Plan

Die Annahmen stehen im Plan, bevor Code entsteht.

Ein falscher Weg ist im Plan billiger korrigiert als nach der Umsetzung.

Der Plan bleibt als Datei liegen: prüfen, teilen, wiederverwenden.

- **Live:** das Rabattmodell im Plan Mode, Claudes Rückfragen, der Plan als Datei

Übung: Ein Feature planen und umsetzen

1. Neue Session starten (Sonnet), mit `Shift+Tab` in den Plan Mode.
2. Weitere Schritte siehe Gist

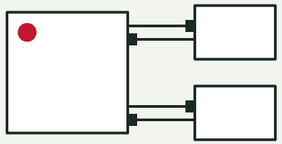
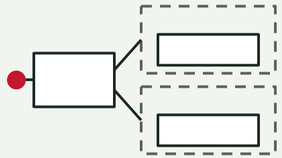
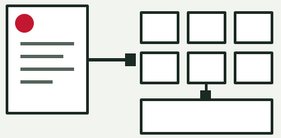
Die Übung im Detail

react.schule/jsdays26-claude

Fertig, wenn `npm test` und `npm run check` grün sind und die Übersicht im Browser steht

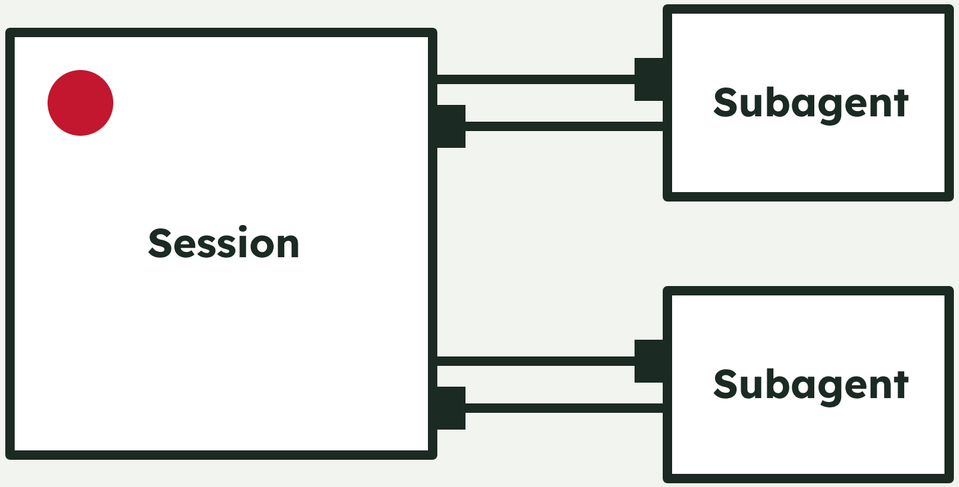
Parallel arbeiten

Vier Wege zur Parallelisierung

		Plan hält	Gut für
	**Subagent**	Claude, Zug um Zug	Suche und Analyse, bei denen nur das Ergebnis zählt
	**Session im Worktree**	ihr	getrennte Pakete mit festem Contract
	**Workflow**	ein Skript	Audits über viele Dateien, große Migrationen

Subagents

Ein Subagent erledigt eine Aufgabe in eigenem Kontext und gibt nur sein Ergebnis zurück.



→ Aufgabe ← Ergebnis

Claude startet Subagents von sich aus oder auf einen Prompt wie „Lass Subagents parallel herausfinden, …“.

Eingebaut sind etwa Explore für die Suche und Plan für den Plan Mode. Eigene liegen unter `.claude/agents/`.

Ein Subagent erbt das Modell der Session, ein eigenes setzt `model` in seiner Definition.

Gut für Suchen und Analysen mit viel Ausgabe, die den Hauptkontext nicht füllen sollen.

Jeder Subagent schickt eigene Anfragen und kostet mit. Höchstens 20 laufen gleichzeitig je Session.

- **Live:** zwei Explore-Agents laufen parallel

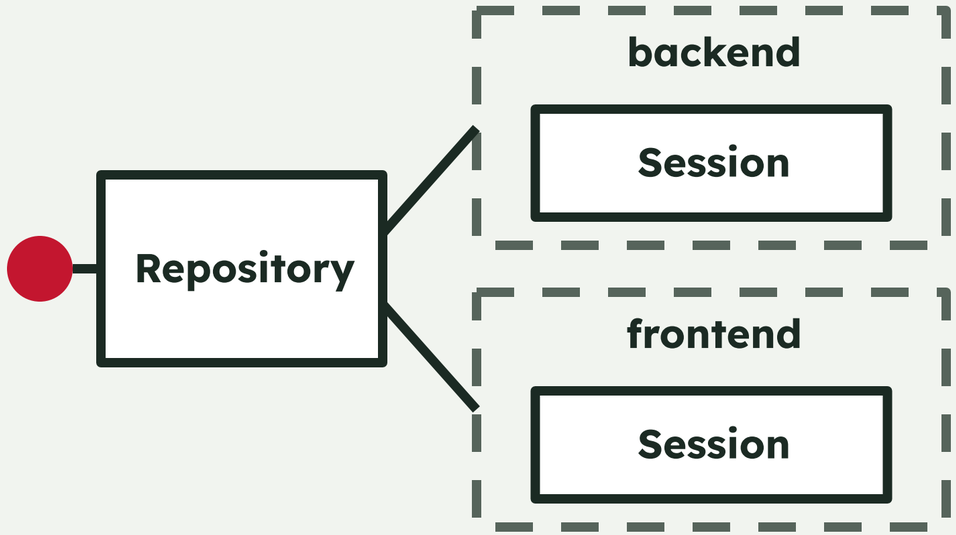
Beispiel: Subagents an den Modulabhängigkeiten

- Worum es geht: Subagents teilen eine Untersuchung auf, die Session bekommt nur ihre Ergebnisse.

- Klon auf main, Sonnet:
  „Lass für jedes Modul unter backend/src einen eigenen Subagent parallel herausfinden, welche anderen Module es importiert und wofür. Fass die Ergebnisse als Abhängigkeiten zwischen den Modulen zusammen und nenn Zyklen. Ändere nichts.“
- Zeigen: mehrere Agents gleichzeitig, die Session bekommt nur Zusammenfassungen.
-

Sessions im Worktree

Mehrere Sessions arbeiten gleichzeitig, jede in eigenem Arbeitsverzeichnis und auf eigenem Branch.




```
claude --worktree backend     # Terminal 1
claude --worktree frontend    # Terminal 2
```

`--worktree` legt den Worktree unter `.claude/worktrees/<name>/` auf einem neuen Branch an und startet Claude darin. Von Hand geht es mit `git worktree add`.

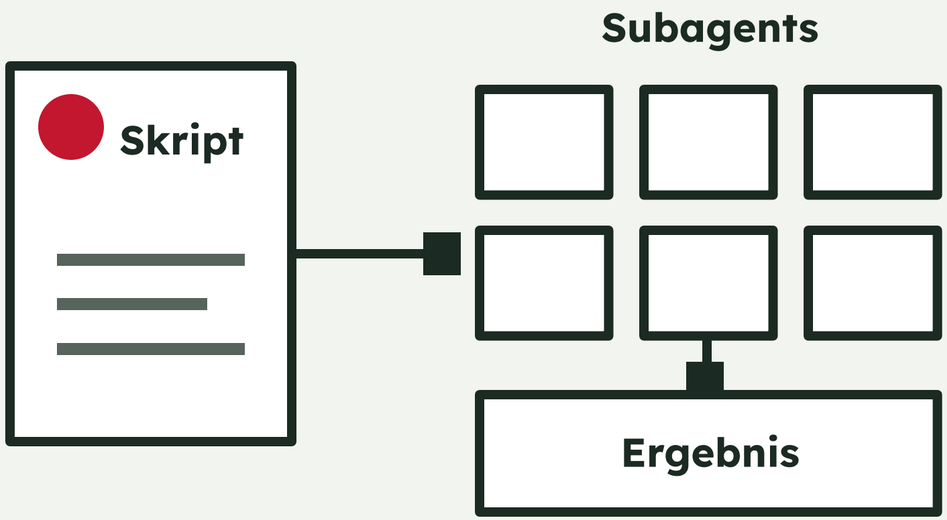
Ein Worktree ist ein frischer Checkout: `npm install` ist nötig, ignorierte Dateien wie `.env` fehlen (`.worktreeinclude` kopiert sie).

Gut für getrennte Pakete, wenn ein Contract vorher feststeht.

Jede Session fasst nur ihr Paket an und prüft mit dem Check ihres Pakets.

Workflows

Ein Workflow ist ein Skript, das Claude schreibt und das viele Subagents steuert.



Anders als im Agent Team hält hier ein Skript den Plan, kein Lead-Agent.

Start mit „ultracode“ oder „use a workflow“ im Prompt. Mitgeliefert ist `/deep-research`.

Das Skript hält Schleife und Zwischenergebnisse. In Claudes Kontext landet nur das Endergebnis.

Ein Workflow läuft im Hintergrund, `/workflows` zeigt den Fortschritt. Ein gelungener Lauf lässt sich als eigenes Kommando speichern.

Gut für Audits über viele Dateien, große Migrationen und Recherche mit gegenseitiger Prüfung.

Ein Lauf braucht deutlich mehr Token. Erst an einem kleinen Ausschnitt testen.

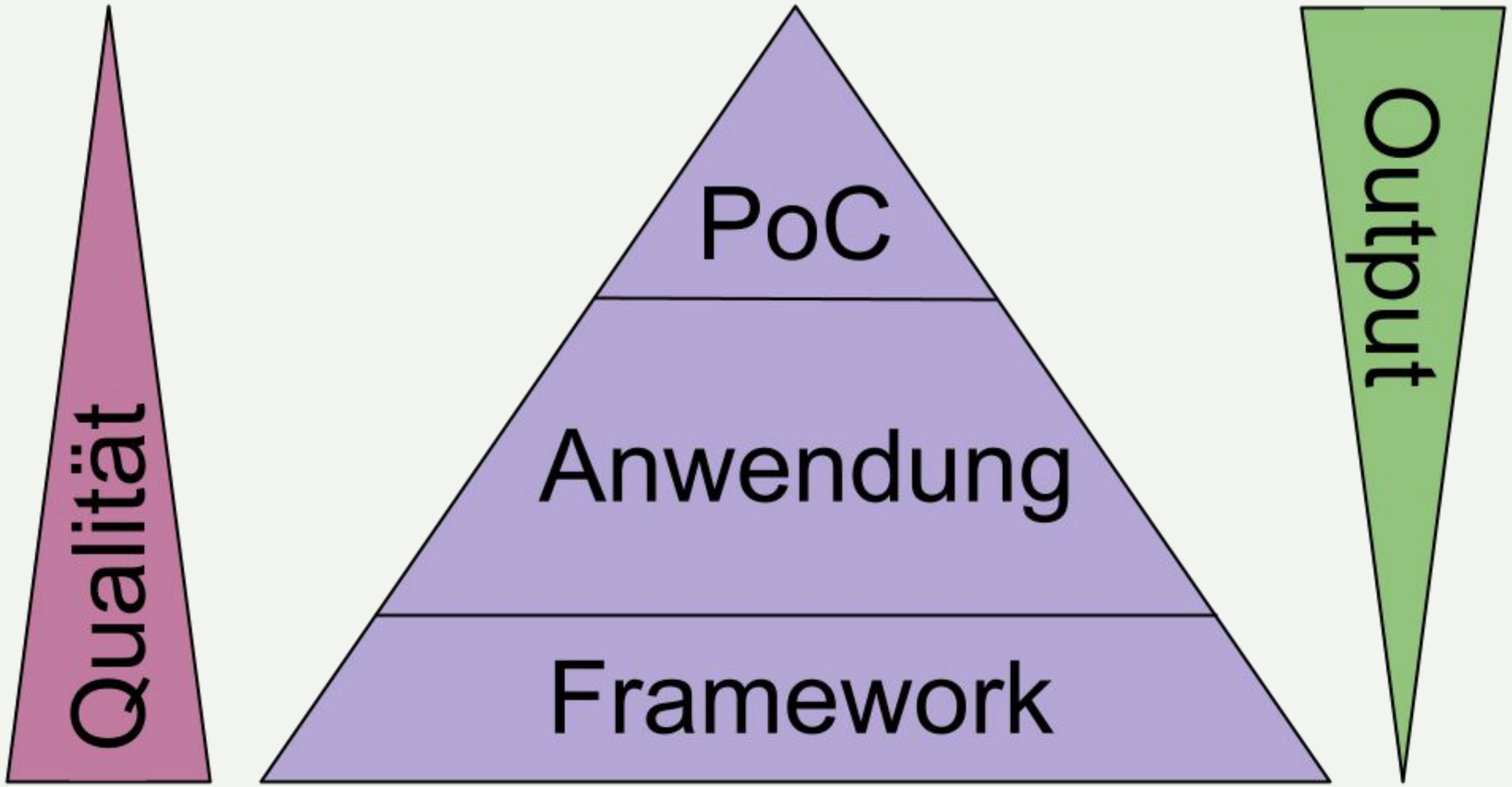
- **Live:** Workflow, der erst Backend und Frontend-Probleme findet, dann Plan macht, dann ausführt

Tests und Vertrauen

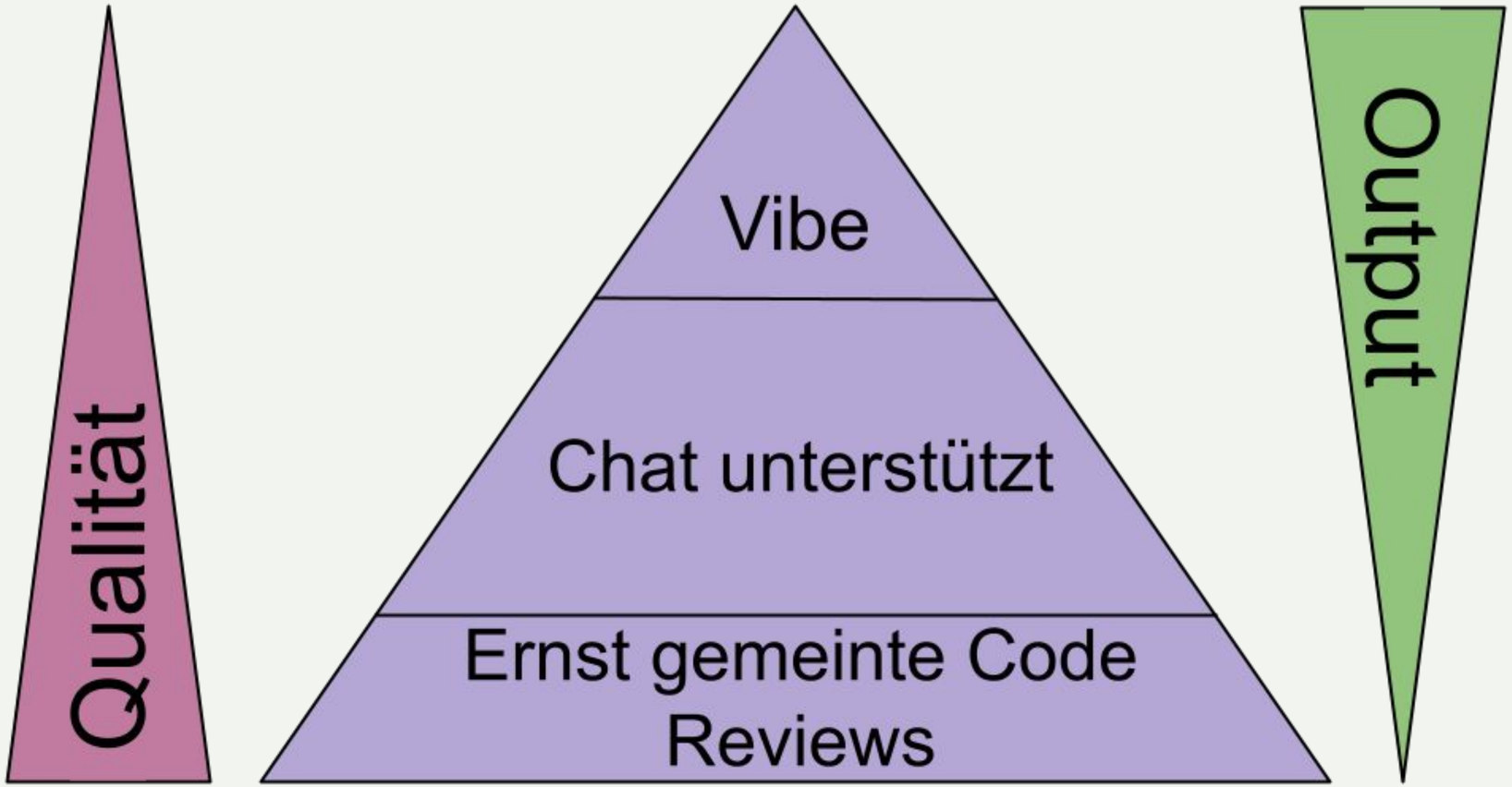


- **Tests: wichtiger? anders? mehr? verlässlich?**
- **Was braucht man, um zu beurteilen, ob Code vertrauenswürdig ist?**
- **Welche Teile des Codes muss man komplett verstehen? Keine?**

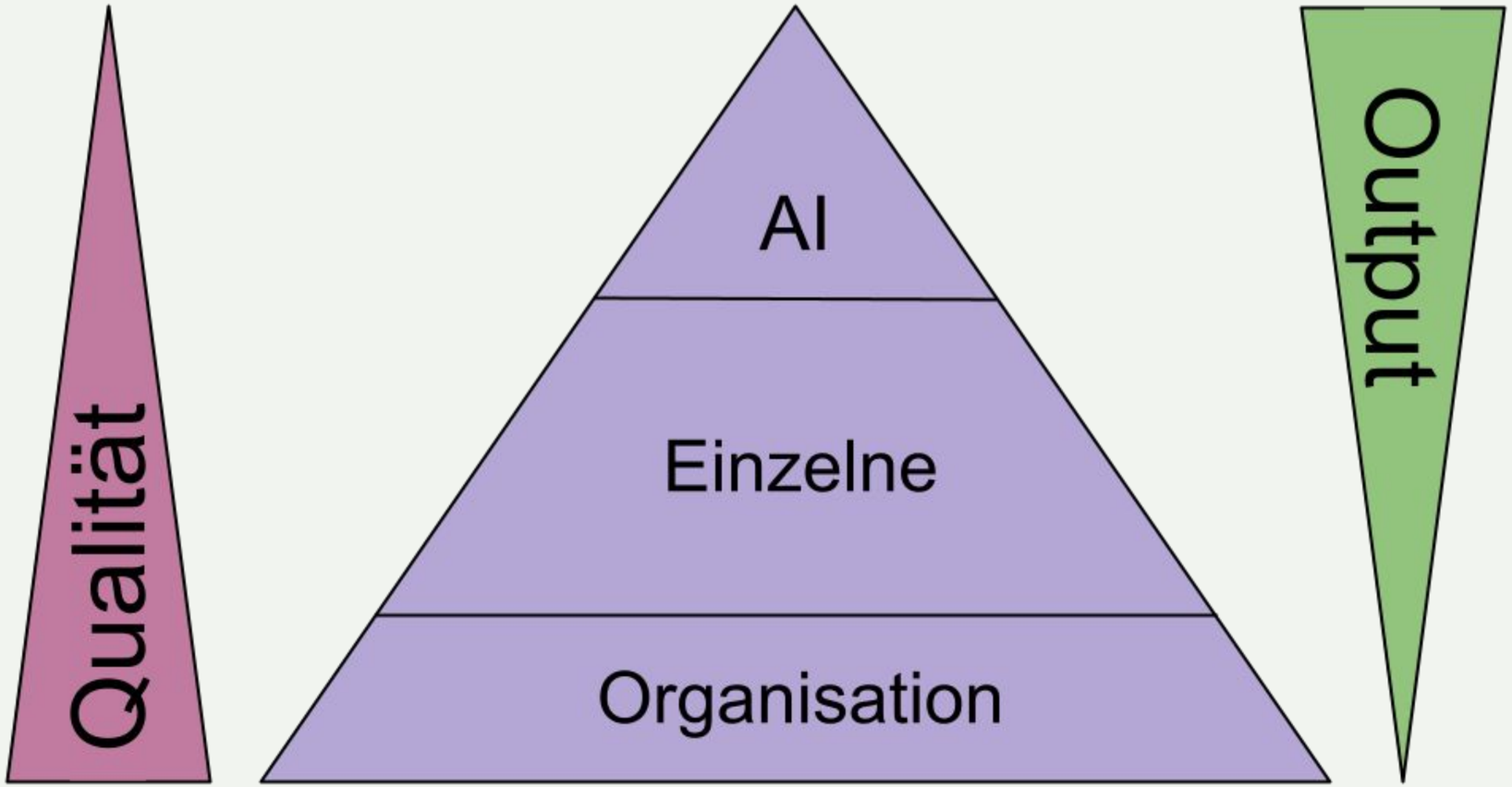
Unterschiedliche Arten von Software



Arten der Entwicklung



Verantwortung für Software



Übung: Testfälle klären, Test zuerst

1. Neue Session starten (Sonnet), mit `Shift+Tab` auf „accept edits“.
2. **Fälle vorschlagen:** Claude schlägt vor, welche Fälle Tests abdecken sollten, und schreibt noch nichts.
3. **Entscheiden:** Ihr legt fest, was gilt: „Rose“ und „rose“? Leerzeichen? Anderer Owner?
4. **Test rot:**Claude schreibt die Tests und zeigt, dass sie fehlschlagen.
5. **Fix:**Erst jetzt wird der Fehler behoben, und die Tests werden grün.

Die Übung im Detail

react.schule/jsdays26-claude

Fertig, wenn die Tests erst rot waren und danach grün sind.

Review

/code-review

`/code-review` sucht Fehler im Diff, ein eigener Prompt prüft, wonach ihr fragt.

/code-review

mitgelieferter Skill

prüft Diff, PR, Branch oder Pfad auf Korrektheitsfehler

Stufen `low` bis `max`, `ultra` : mehrere Agents in der Cloud

`--fix` wendet Befunde an, `--comment` schreibt an den PR

Eigener Prompt

eure Regeln aus der `CLAUDE.md`

Architektur und Konventionen

Testlücken

ein bestimmter Schwerpunkt

`/security-review` sucht gezielt nach Sicherheitslücken.

- **Live:** `/code-review` startet am Beamer, parallel zur folgenden Übung

Übung: Den eigenen Stand prüfen lassen

1. Alles committen. Frische Session starten (Sonnet), mit `Shift+Tab` in den manuellen Modus.

2. **In Claude Code eingeben**

   > Prüfe den Diff von v1.0 zu main. Zwei Schwerpunkte: Hält er die Regeln aus der CLAUDE.md ein? Und was ist an neuem Verhalten nicht durch Tests gedeckt? Ändere nichts, nenn mir die Befunde.

Die Übung im Detail

react.schule/jsdays26-claude

Fertig, wenn ihr die zwei wichtigsten Befunde im Code nachvollzogen habt.

Claude anpassen

Claude anpassen

Claude Code lässt sich auf vielen Ebenen anpassen, und beim Anpassen hilft Claude selbst.

Was	Wo	Wofür
`CLAUDE.md`	im Projekt, `~/.claude/CLAUDE.md`	Regeln und Konventionen, immer im Kontext
Settings	`.claude/settings.json`	Modell, Permissions, Umgebung, Sandbox
Skills	`.claude/skills/<name>/SKILL.md`	Anleitungen, die erst bei Bedarf laden
Subagents	`.claude/agents/`	Helfer mit eigenem Kontext



Skills

Ein Skill ist eine Anleitung, die Claude erst lädt, wenn sie gebraucht wird.

`.claude/skills/review/SKILL.md`

```
---
name: review
description: Prüft den Diff
  gegen die CLAUDE.md
---
Prüfe den Diff von v1.0
zu main ...
```

Die Datei liegt im Projekt unter `.claude/skills/` oder für alle Projekte unter `~/.claude/skills/`.

Anhand der `description` erkennt Claude, wann der Skill passt. Direkt aufrufen lässt er sich mit `/review`.

Die `CLAUDE.md` ist immer im Kontext, ein Skill nur bei Bedarf.

`/skills` listet alle Skills.

Claude anpassen

Was	Wo	Wofür
Hooks	in der `settings.json`	Skripte an festen Punkten, setzen Regeln durch
MCP-Server	`.mcp.json`, `claude mcp add`	Werkzeuge für GitHub, Jira und andere
Plugins	`/plugin`	Pakete aus Skills, Agents, Hooks und MCP
Anzeige	`/statusline`, `/output-style`	Statuszeile und Stil der Antworten



Beim Anpassen hilft Claude selbst

Tipp: Fragt Claude, wie etwas in Claude Code geht, und lasst es einrichten:

Prompt für Claude Code

> **Richte einen Hook ein, der nach jeder Änderung die Typen prüft.**

- **Live:** Claude fragen, wie man eine Statuszeile mit Kosten einrichtet

Skills

Ein Skill ist eine Anleitung, die Claude erst lädt, wenn sie gebraucht wird.

`.claude/skills/review/SKILL.md`

```
---
name: review
description: Prüft den Diff
  gegen die CLAUDE.md
---
Prüfe den Diff von v1.0
zu main ...
```

Die Datei liegt im Projekt unter `.claude/skills/` oder für alle Projekte unter `~/.claude/skills/`.

Anhand der `description` erkennt Claude, wann der Skill passt. Direkt aufrufen lässt er sich mit `/review`.

Die `CLAUDE.md` ist immer im Kontext, ein Skill nur bei Bedarf.

`/skills` listet alle Skills.

- **Live:** den Review-Skill aufrufen

Vielen Dank und viel Spaß!

